

Lab 4: Regression and Classification Evaluation Metrics

Aim

To implement the **K-Nearest Neighbours (KNN)** classification algorithm on the **Breast Cancer Wisconsin Diagnostic Dataset**, analyze its performance using different validation techniques and evaluate the model using classification metrics. The lab also compares these metrics with regression evaluation metrics studied in **Linear Regression (Lab 3)**.

Objectives

After completing this lab, you should be able to:

- Understand how the KNN classification algorithm works.
 - Prepare datasets for machine learning.
 - Apply feature scaling using `StandardScaler`.
 - Perform different train-test splits (80:20, 70:30, 90:10).
 - Select an appropriate value of **K** using heuristic methods ($K = \sqrt{n}$).
 - Evaluate model performance using Cross Validation (5-Fold and 10-Fold).
 - Interpret classification evaluation metrics (Accuracy, Precision, Recall, F1 Score).
 - Understand ROC Curve and ROC-AUC.
 - Compare regression evaluation metrics with classification metrics.
 - Analyze the advantages and limitations of KNN.
-

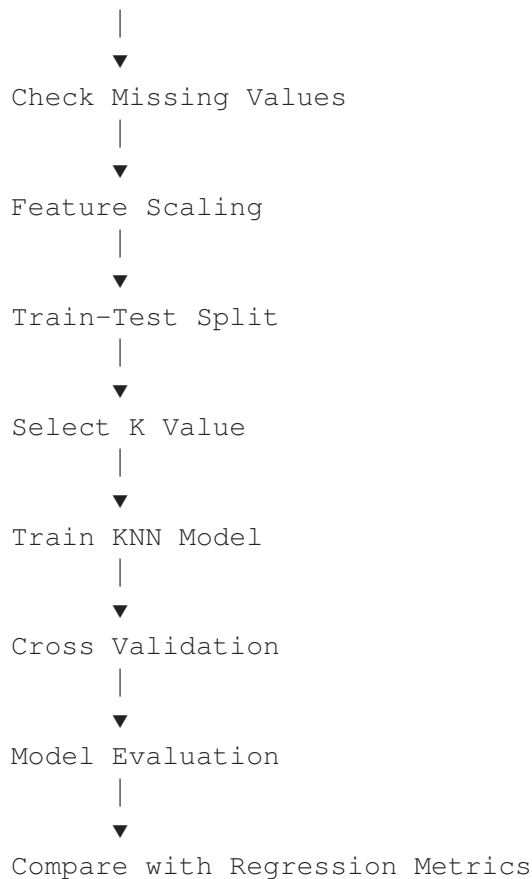
Dataset Information

Dataset: Breast Cancer Wisconsin (Diagnostic)

- **Total Samples:** 569
 - **Features:** 30 Numerical Features
 - **Target Variable:**
 - 0 → Malignant
 - 1 → Benign
-

Machine Learning Workflow





Task 1 — Data Preparation

Why Feature Scaling?

KNN calculations are based on distance metrics (such as Euclidean or Manhattan distance) between data points. Features with larger numerical ranges will dominate these distance calculations, rendering features with smaller ranges ineffective.

To address this, we apply feature scaling using `StandardScaler`. This transforms the distribution of each feature to have:

- **Mean = 0**
- **Standard Deviation = 1**

This ensures that every feature contributes equally to the distance calculation.

Mapping Target Variable

The target column in the raw dataset contains string values:

- `"M"` for Malignant (predicted class `0` in our setup)
- `"B"` for Benign (predicted class `1` in our setup)

We will map these to numerical values to make them compatible with machine learning libraries like scikit-learn.

```

In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score
import os

# Set up directories and aesthetics
os.makedirs('../images', exist_ok=True)
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (8, 6)
plt.rcParams['font.size'] = 10

# Load dataset
# The dataset has an index in the first column, which we read using index_col=0
df = pd.read_csv('../data/breast_cancer.csv', index_col=0)

print("--- Dataset Shape & Information ---")
print(f"Shape: {df.shape[0]} rows, {df.shape[1]} columns")
print("\nFirst 5 Rows:")
display(df.head())

print("\n--- Data Types and Non-Null Counts ---")
df.info()

print("\n--- Checking for Missing Values & Duplicates ---")
missing_values = df.isnull().sum().sum()
duplicate_rows = df.duplicated().sum()
print(f"Total Missing Values: {missing_values}")
print(f"Total Duplicate Rows: {duplicate_rows}")

print("\n--- Target Variable Distribution ---")
class_counts = df['y'].value_counts()
class_props = df['y'].value_counts(normalize=True)
for cls, count in class_counts.items():
    print(f"Class '{cls}': {count} samples ({class_props[cls]*100:.2f}%)"

# Map Target Variable 'y' ('M' -> 0, 'B' -> 1)
df['y'] = df['y'].map({'M': 0, 'B': 1})
print("\nTarget Variable 'y' mapped successfully:")
print(df['y'].value_counts())

```

--- Dataset Shape & Information ---

Shape: 569 rows, 31 columns

First 5 Rows:

	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean	x.smoothness_mean
1	13.540	14.36	87.46	566.3	0.09779
2	13.080	15.71	85.63	520.0	0.10750
3	9.504	12.44	60.34	273.9	0.10240
4	13.030	18.42	82.61	523.8	0.08983
5	8.196	16.84	51.71	201.9	0.08600

5 rows × 31 columns

```

--- Data Types and Non-Null Counts ---
<class 'pandas.DataFrame'>
RangeIndex: 569 entries, 1 to 569
Data columns (total 31 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   x.radius_mean                        569 non-null    float64
1   x.texture_mean                      569 non-null    float64
2   x.perimeter_mean                   569 non-null    float64
3   x.area_mean                        569 non-null    float64
4   x.smoothness_mean                  569 non-null    float64
5   x.compactness_mean                 569 non-null    float64
6   x.concavity_mean                   569 non-null    float64
7   x.concave_pts_mean                 569 non-null    float64
8   x.symmetry_mean                    569 non-null    float64
9   x.fractal_dim_mean                 569 non-null    float64
10  x.radius_se                         569 non-null    float64
11  x.texture_se                        569 non-null    float64
12  x.perimeter_se                     569 non-null    float64
13  x.area_se                          569 non-null    float64
14  x.smoothness_se                    569 non-null    float64
15  x.compactness_se                   569 non-null    float64
16  x.concavity_se                     569 non-null    float64
17  x.concave_pts_se                   569 non-null    float64
18  x.symmetry_se                      569 non-null    float64
19  x.fractal_dim_se                   569 non-null    float64
20  x.radius_worst                     569 non-null    float64
21  x.texture_worst                    569 non-null    float64
22  x.perimeter_worst                  569 non-null    float64
23  x.area_worst                       569 non-null    float64
24  x.smoothness_worst                 569 non-null    float64
25  x.compactness_worst                569 non-null    float64
26  x.concavity_worst                  569 non-null    float64
27  x.concave_pts_worst                569 non-null    float64
28  x.symmetry_worst                   569 non-null    float64
29  x.fractal_dim_worst                569 non-null    float64
30  y                                  569 non-null    str
dtypes: float64(30), str(1)
memory usage: 137.9 KB

```

```

--- Checking for Missing Values & Duplicates ---
Total Missing Values: 0
Total Duplicate Rows: 0

```

```

--- Target Variable Distribution ---
Class 'B': 357 samples (62.74%)
Class 'M': 212 samples (37.26%)

```

```

Target Variable 'y' mapped successfully:
y
1    357
0    212
Name: count, dtype: int64

```

Task 2 — Train-Test Split Analysis

We perform three different train-test splits to evaluate how training set size affects the model's performance.

- **80:20 Split**
- **70:30 Split**
- **90:10 Split**

Critical Concept: Avoiding Data Leakage in Feature Scaling

To evaluate the model's true generalization capacity, we must split the data **before** feature scaling.

- The `StandardScaler` must fit its parameters (μ and σ) **only** on the training features.
- We then transform both the training features and the testing features using these fitted parameters. If we scale the entire dataset before splitting, information from the test set (mean and variance) leaks into the training set, causing **data leakage** and leading to overly optimistic test performance estimates.

```
In [ ]: # Separate features and target
X = df.drop(columns=['y'])
y = df['y']

# Define splits to analyze
split_ratios = {
    '80:20': 0.20,
    '70:30': 0.30,
    '90:10': 0.10
}

split_results = []

for split_name, test_size in split_ratios.items():
    # Stratify split ensures that class ratios are preserved in train and test
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=42, stratify=y
    )

    # Scale features (Fit only on training, transform both)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Heuristic K = sqrt(n_train), rounded to the nearest odd integer
    n_train = len(X_train)
    heuristic_k = int(np.round(np.sqrt(n_train)))
    if heuristic_k % 2 == 0:
        heuristic_k += 1 # Make odd to prevent ties

    # Fit KNN Model
    knn = KNeighborsClassifier(n_neighbors=heuristic_k)
    knn.fit(X_train_scaled, y_train)
```

```

# Evaluate
y_pred = knn.predict(X_test_scaled)
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

split_results.append({
    'Split Ratio': split_name,
    'Train Size (n)': n_train,
    'Test Size': len(X_test),
    'Heuristic K': heuristic_k,
    'Accuracy': acc,
    'Precision': prec,
    'Recall': rec,
    'F1 Score': f1
})

# Compile results in a table
df_splits = pd.DataFrame(split_results)
print("--- Comparison of Different Train-Test Splits ---")
display(df_splits)

```

--- Comparison of Different Train-Test Splits ---

	Split Ratio	Train Size (n)	Test Size	Heuristic K	Accuracy	Precision	Recall	F1 Score
0	80:20	455	114	21	0.964912	0.947368	1.0	0.972973
1	70:30	398	171	21	0.947368	0.922414	1.0	0.959641
2	90:10	512	57	23	0.982456	0.972973	1.0	0.986301

Analysis: How Train-Test Split Affects Performance & Stability

1. 90:10 Split:

- **Generalization:** With 90% training data, the model sees a larger variety of examples allowing it to potentially capture the feature space more accurately.
- **Stability:** However, the test size is very small (10% or 57 samples). This makes the evaluation metric highly sensitive to individual test samples. An error on a single test sample changes the accuracy by nearly 1.8%, leading to high variance in the reported metrics.

2. 70:30 Split:

- **Generalization:** The model is trained on fewer samples (398 samples). It might not represent all clinical patterns as comprehensively, potentially limiting its training capacity.
- **Stability:** The test set size is larger (171 samples), which yields a highly reliable, low-variance evaluation of the trained model.

3. 80:20 Split:

- This split provides a balanced trade-off, offering sufficient training data (455 samples) for high generalization while maintaining a large enough test set (114 samples) for reliable evaluation.

Task 3 — KNN Model Optimization

Step 1: Heuristic K Selection

We use the rule of thumb $K = \sqrt{n}$ where n is the number of training samples. For standard 80:20 split: $n = 455 \implies K = \sqrt{455} \approx 21.33 \implies K \approx 21$. To avoid voting ties, we adjust K to the nearest odd integer.

Step 2: Experiment with Different K Values

Instead of relying solely on the heuristic, we evaluate a range of K values around the heuristic ($K-5$ to $K+5$, which is 16 to 26), and broaden it to $K \in [1, 35]$ to visualize the trend.

```
In [ ]: # Standardize 80:20 split for optimization
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

k_values = list(range(1, 36))
train accuracies = []
test accuracies = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)

    train accuracies.append(accuracy_score(y_train, knn.predict(X_train_scaled)))
    test accuracies.append(accuracy_score(y_test, knn.predict(X_test_scaled)))

# Find best K
best_idx = np.argmax(test accuracies)
best_k = k_values[best_idx]
best_test_acc = test accuracies[best_idx]

print(f"Heuristic K: 21 (Test Accuracy: {test accuracies[k_values.index(21)]})")
print(f"Optimal K from Parameter Search: {best_k} (Test Accuracy: {best_test_acc})")

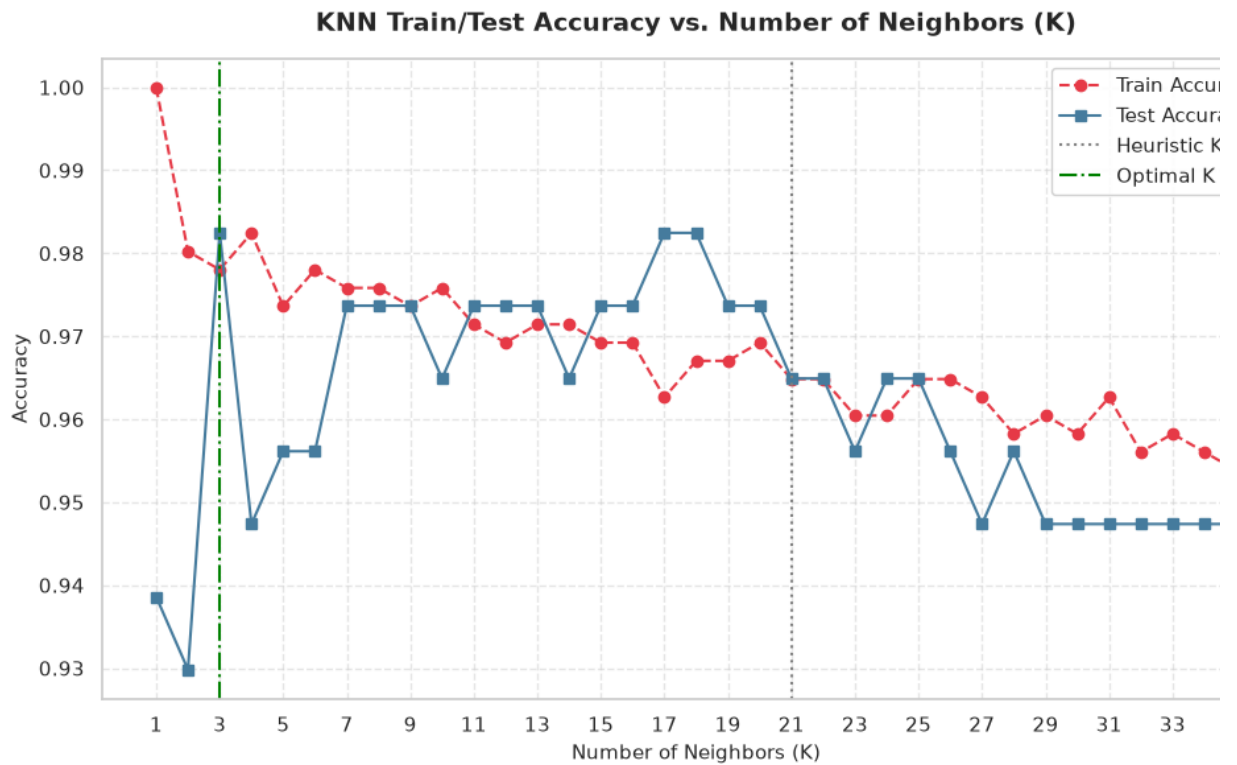
# Plot Train & Test Accuracy vs K
plt.figure(figsize=(10, 6), dpi=100)
plt.plot(k_values, train accuracies, label='Train Accuracy', color='#E63976')
plt.plot(k_values, test accuracies, label='Test Accuracy', color='#457B9D')
plt.axvline(x=21, color='gray', linestyle=':', label='Heuristic K (21)')
plt.axvline(x=best_k, color='green', linestyle='-.', label=f'Optimal K ({best_k})')

plt.title('KNN Train/Test Accuracy vs. Number of Neighbors (K)', fontsize=12)
plt.xlabel('Number of Neighbors (K)', fontsize=11)
plt.ylabel('Accuracy', fontsize=11)
plt.xticks(k_values[::2])
```

```
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(frameon=True, facecolor='white', framealpha=0.9)
plt.tight_layout()
plt.savefig('../images/accuracy_vs_k.png', dpi=150)
plt.show()
```

Heuristic K: 21 (Test Accuracy: 96.49%)

Optimal K from Parameter Search: 3 (Test Accuracy: 98.25%)



Distance Metrics in KNN

The selection of the distance metric is critical to define the "closeness" of neighbors.

1. **Euclidean Distance (L2 Norm):** $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^D (x_i - y_i)^2}$

- Suitable for continuous features and represents the straight-line geometric distance.
- Requires feature scaling as it is highly sensitive to features with large variances.

2. **Manhattan Distance (L1 Norm):** $d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^D |x_i - y_i|$

- Measures grid-like distance (distance along axes).
- More robust to outliers and preferred in high-dimensional sparse datasets.

```
In [ ]: # Compare distance metrics with the optimal K value
metrics_comp = []

for metric_name in ['euclidean', 'manhattan']:
    knn = KNeighborsClassifier(n_neighbors=best_k, metric=metric_name)
    knn.fit(X_train_scaled, y_train)
    y_pred = knn.predict(X_test_scaled)

    metrics_comp.append({
        'Distance Metric': metric_name.capitalize(),
        'Accuracy': accuracy_score(y_test, y_pred),
    })
```

```

        'Precision': precision_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred),
        'F1 Score': f1_score(y_test, y_pred)
    })

df_metrics = pd.DataFrame(metrics_comp)
print("--- Distance Metric Comparison Table ---")
display(df_metrics)

```

--- Distance Metric Comparison Table ---

	Distance Metric	Accuracy	Precision	Recall	F1 Score
0	Euclidean	0.982456	0.972973	1.000000	0.986301
1	Manhattan	0.964912	0.959459	0.986111	0.972603

Decision Boundary Visualization

Since our dataset has 30 features, visualizing decision boundaries directly is impossible. To overcome this, we project the scaled training features into a 2D space using **Principal Component Analysis (PCA)**.

We then fit KNN classifiers on the 2D projected space for $K \in \{1, 5, 10, 20\}$ and plot the resulting decision boundaries to observe:

- **Small K (e.g. $K=1$):** High variance, complex, noisy boundaries that closely wrap around points (Overfitting).
- **Large K (e.g. $K=20$):** High bias, extremely smooth and simple boundaries (Underfitting).

```

In [ ]: from sklearn.decomposition import PCA
        from matplotlib.colors import ListedColormap

        # Project scaled features to 2D using PCA
        pca = PCA(n_components=2)
        X_train_pca = pca.fit_transform(X_train_scaled)

        k_list = [1, 5, 10, 20]
        cmap_light = ListedColormap(['#FFCECE', '#E6F9EC']) # Very light red and light green
        cmap_bold = ['#E63946', '#2A9D8F'] # Professional Crimson and Teal

        for K in k_list:
            knn = KNeighborsClassifier(n_neighbors=K)
            knn.fit(X_train_pca, y_train)

            # Setup boundary mesh grid
            x_min, x_max = X_train_pca[:, 0].min() - 1, X_train_pca[:, 0].max() + 1
            y_min, y_max = X_train_pca[:, 1].min() - 1, X_train_pca[:, 1].max() + 1
            xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.05),
                                np.arange(y_min, y_max, 0.05))

            Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
            Z = Z.reshape(xx.shape)

            plt.figure(figsize=(8, 6), dpi=100)
            plt.contourf(xx, yy, Z, cmap=cmap_light, alpha=0.8)

```

```

# Scatter plot
scatter = plt.scatter(X_train_pca[:, 0], X_train_pca[:, 1], c=y_train,
                      cmap=ListedColormap(cmap_bold), edgecolor='k',

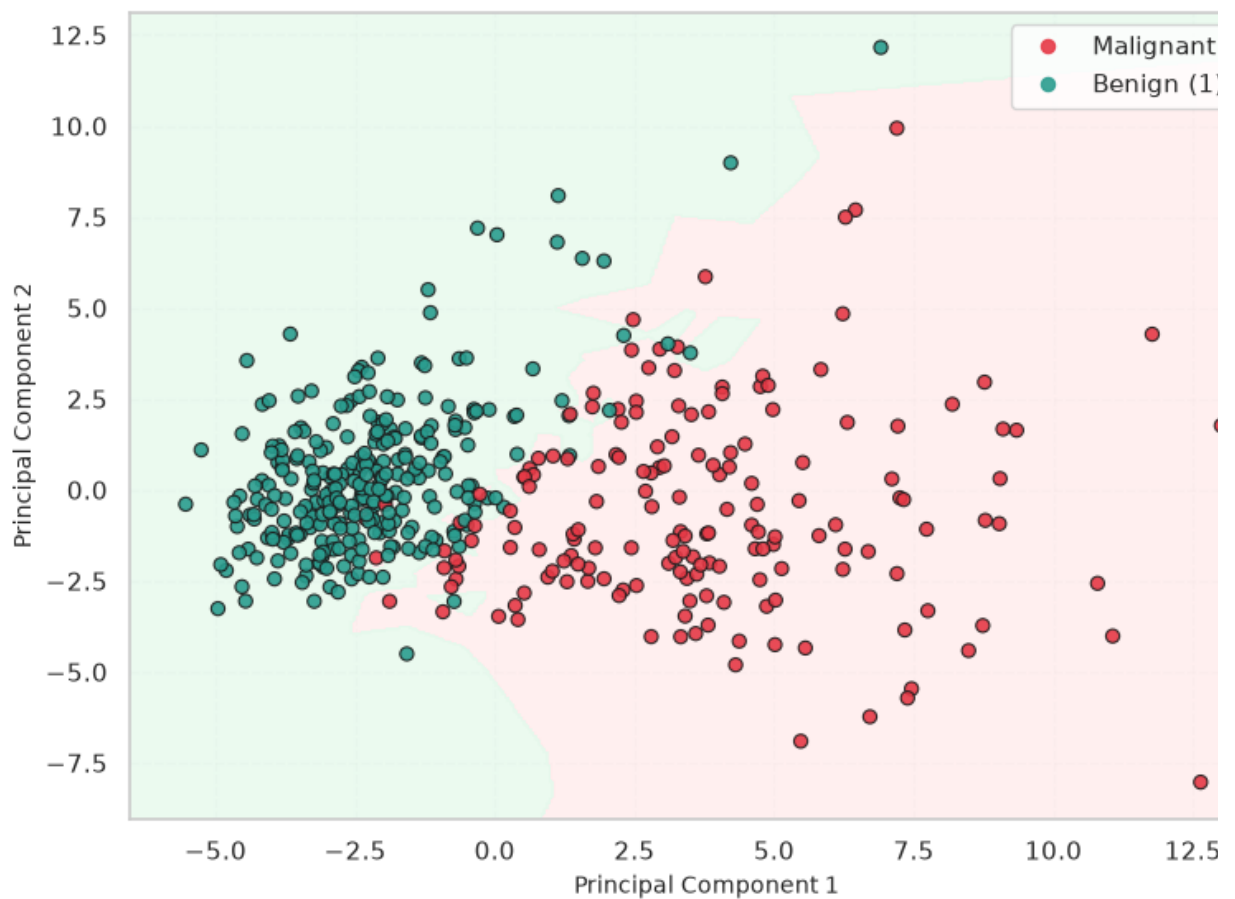
plt.title(f'KNN Decision Boundary (K = {K}) in PCA-Projected 2D Space')
plt.xlabel('Principal Component 1', fontsize=10)
plt.ylabel('Principal Component 2', fontsize=10)

# Legend
handles, labels = scatter.legend_elements()
plt.legend(handles, ['Malignant (0)', 'Benign (1)'], loc='upper right')
plt.grid(True, linestyle='--', alpha=0.3)
plt.tight_layout()

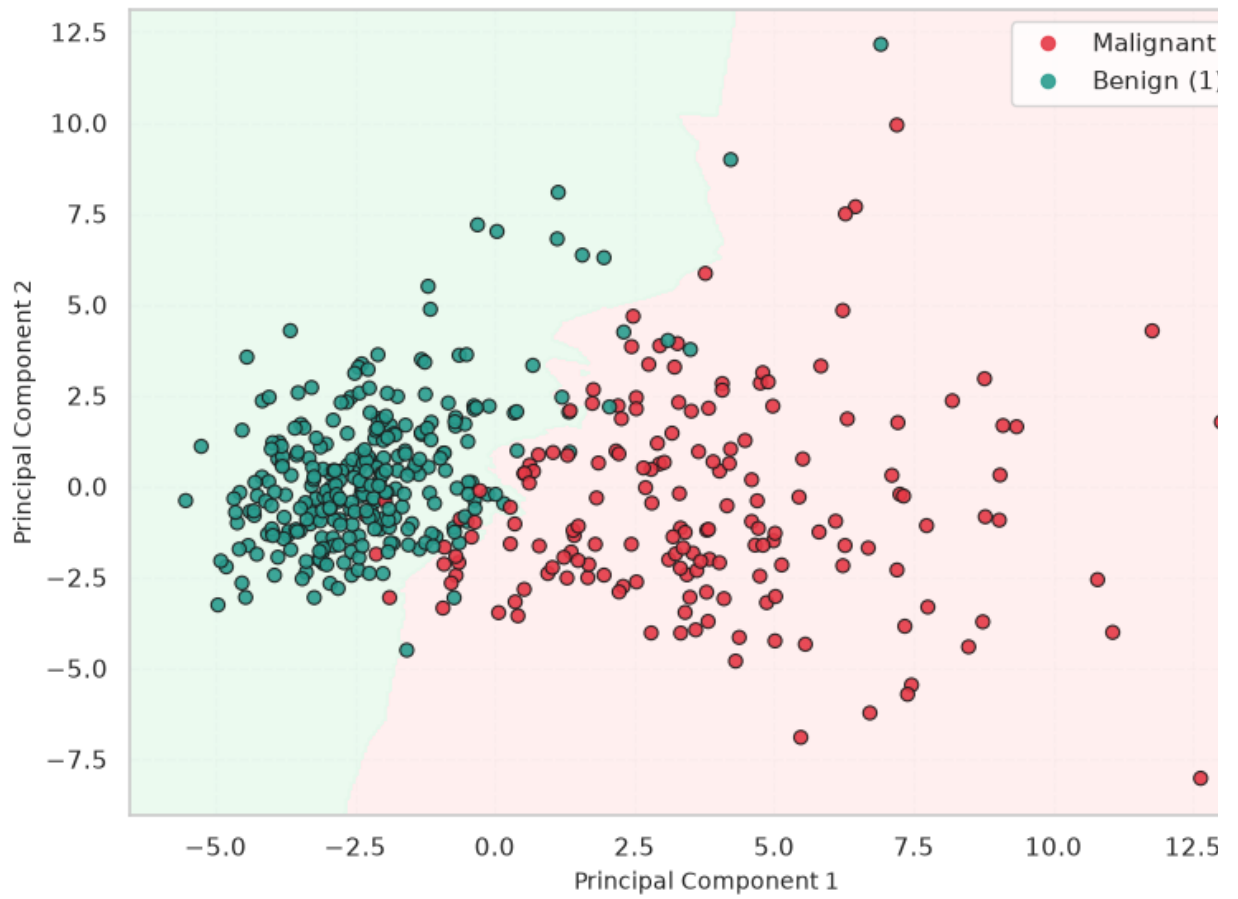
# Save image
plt.savefig(f'../images/decision_boundary_k{K}.png', dpi=150)
plt.show()

```

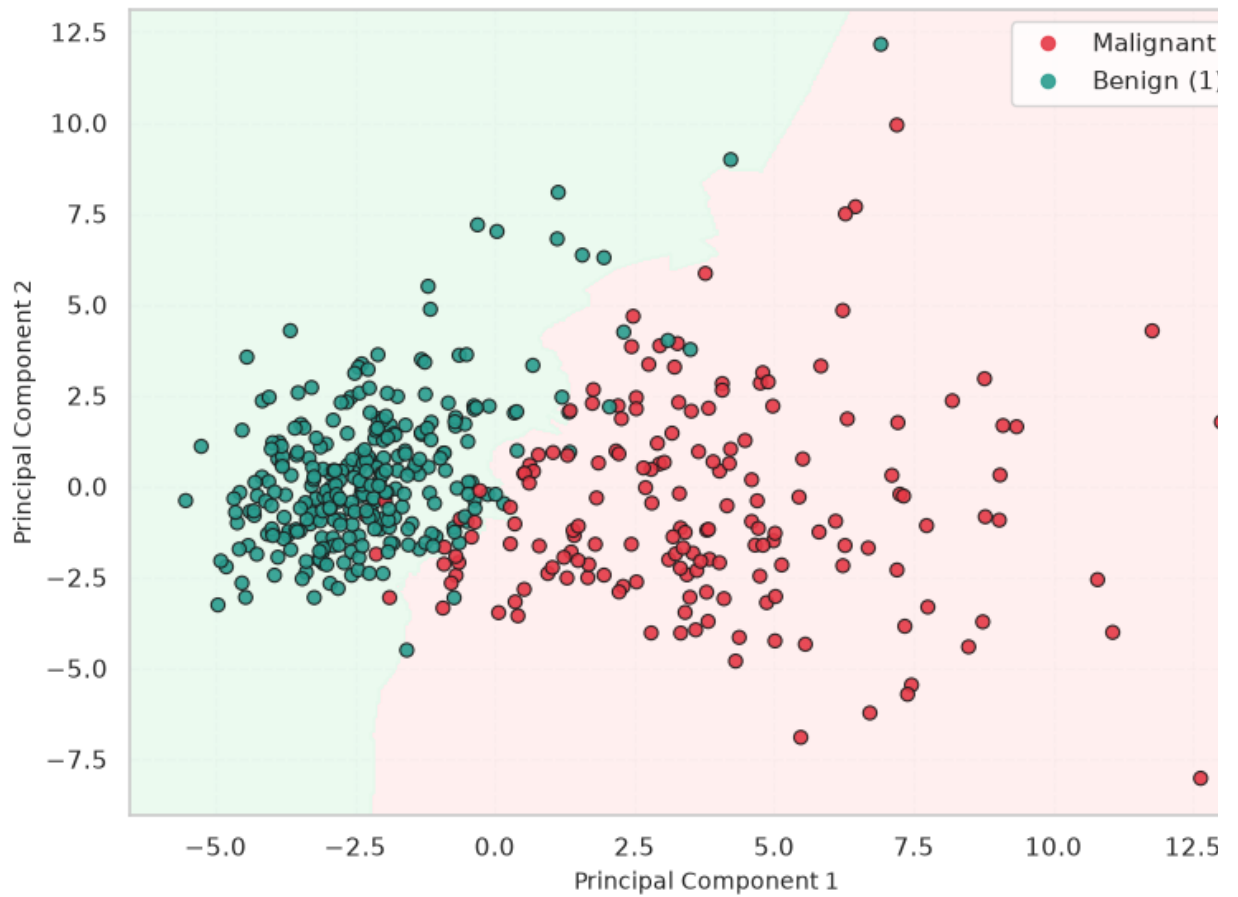
KNN Decision Boundary (K = 1) in PCA-Projected 2D Space

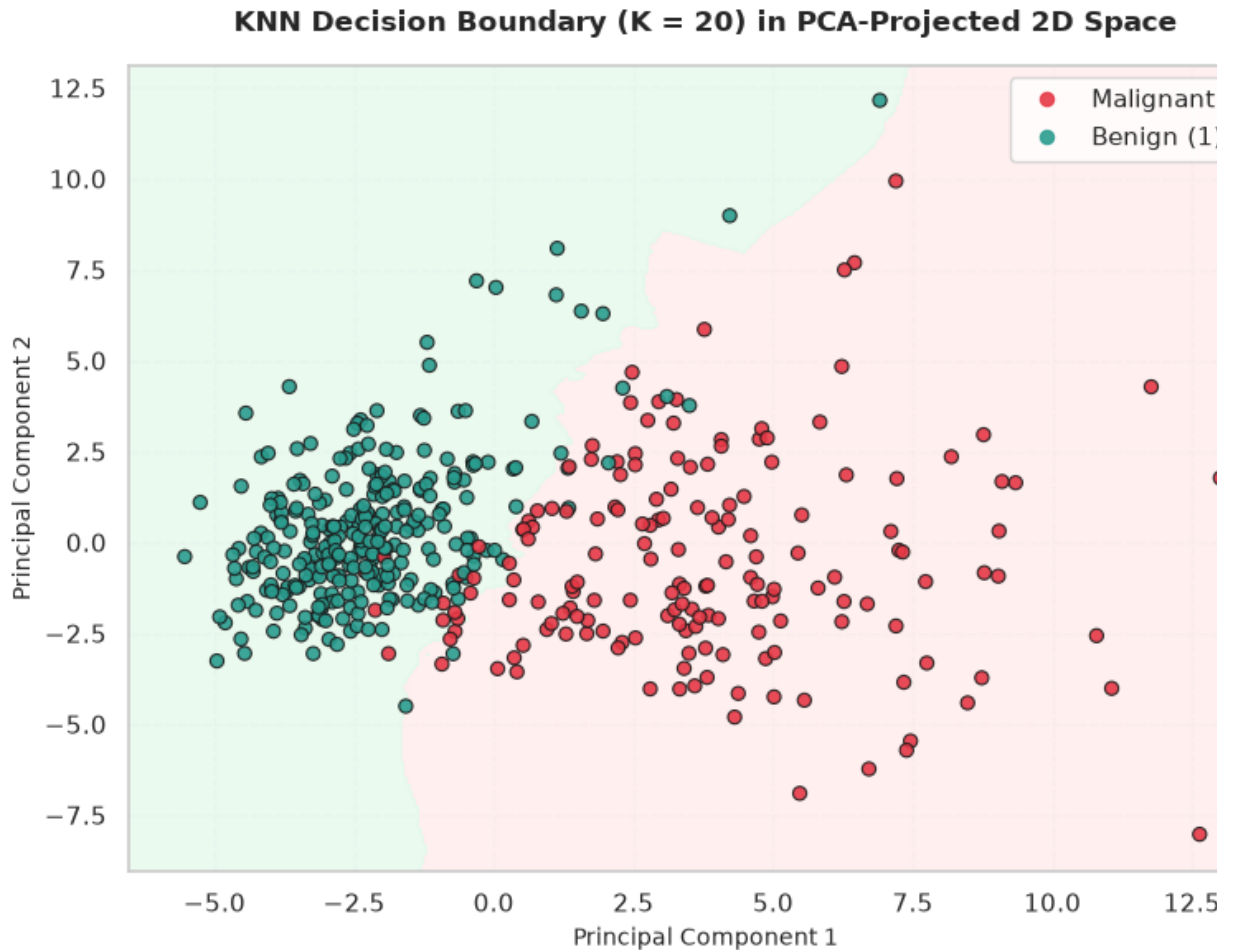


KNN Decision Boundary (K = 5) in PCA-Projected 2D Space



KNN Decision Boundary (K = 10) in PCA-Projected 2D Space





Task 4 — Cross Validation

To ensure our evaluation is not biased by a single split, we perform **5-Fold** and **10-Fold Cross Validation**.

Preventing Data Leakage in Cross Validation

When validating, the scaling step must occur inside each fold independently. Fitting a scaler on the entire dataset and then cross-validating results in data leakage. To avoid this, we use scikit-learn's `Pipeline` which automatically scales the training subset of each fold and applies the transformation to the validation fold.

```
In [ ]: from sklearn.pipeline import Pipeline

cv_results = {}

for folds in [5, 10]:
    # Setup pipeline with scaling and model
    pipeline = Pipeline([
        ('scaler', StandardScaler()),
        ('knn', KNeighborsClassifier(n_neighbors=best_k))
    ])

    scores = cross_val_score(pipeline, X, y, cv=folds, scoring='accuracy')
    cv_results[f'{folds}-Fold'] = scores
    print(f"{folds}-Fold Cross Validation Mean Accuracy: {scores.mean()*100}")
```

```
# Compare with simple train-test split accuracy
print(f"Simple 80:20 Split Test Accuracy (Optimal K = {best_k}): {best_te
```

5-Fold Cross Validation Mean Accuracy: 95.95% ($\pm 1.91\%$)
 10-Fold Cross Validation Mean Accuracy: 96.65% ($\pm 2.16\%$)
 Simple 80:20 Split Test Accuracy (Optimal K = 3): 98.25%

Task 5 — Model Evaluation

We evaluate our final model (using the optimal K found from parameter search on the 8 split) using multiple classification evaluation metrics:

- **Accuracy:** General correctness.
- **Precision:** $\frac{TP}{TP + FP}$ - Proportion of predicted positives that are actually positive (critical when false positives are costly).
- **Recall (Sensitivity):** $\frac{TP}{TP + FN}$ - Proportion of actual positives correctly identified (extremely critical in cancer diagnosis to avoid missing malignant cases).
- **F1 Score:** Harmonic mean of Precision and Recall.
- **Confusion Matrix:** Map of TP, TN, FP, FN.
- **ROC Curve & ROC-AUC:** Graphical representation of TPR vs FPR across all thresholds.

```
In [ ]: # Fit the final model with optimal K
final_knn = KNeighborsClassifier(n_neighbors=best_k)
final_knn.fit(X_train_scaled, y_train)

y_pred = final_knn.predict(X_test_scaled)
y_pred_proba = final_knn.predict_proba(X_test_scaled)[:, 1]

# 1. Classification Report
print("--- Classification Report ---")
print(classification_report(y_test, y_pred, target_names=['Malignant (0)', 'Benign (1)']))

# 2. Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5), dpi=100)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Malignant (0)', 'Benign (1)'],
            yticklabels=['Malignant (0)', 'Benign (1)'],
            annot_kws={'size': 14, 'weight': 'bold'})
plt.title('Confusion Matrix', fontsize=14, fontweight='bold', pad=15)
plt.xlabel('Predicted Label', fontsize=11)
plt.ylabel('True Label', fontsize=11)
plt.tight_layout()
plt.savefig('../images/confusion_matrix.png', dpi=150)
plt.show()

# 3. ROC Curve and ROC-AUC
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
roc_auc = auc(fpr, tpr)

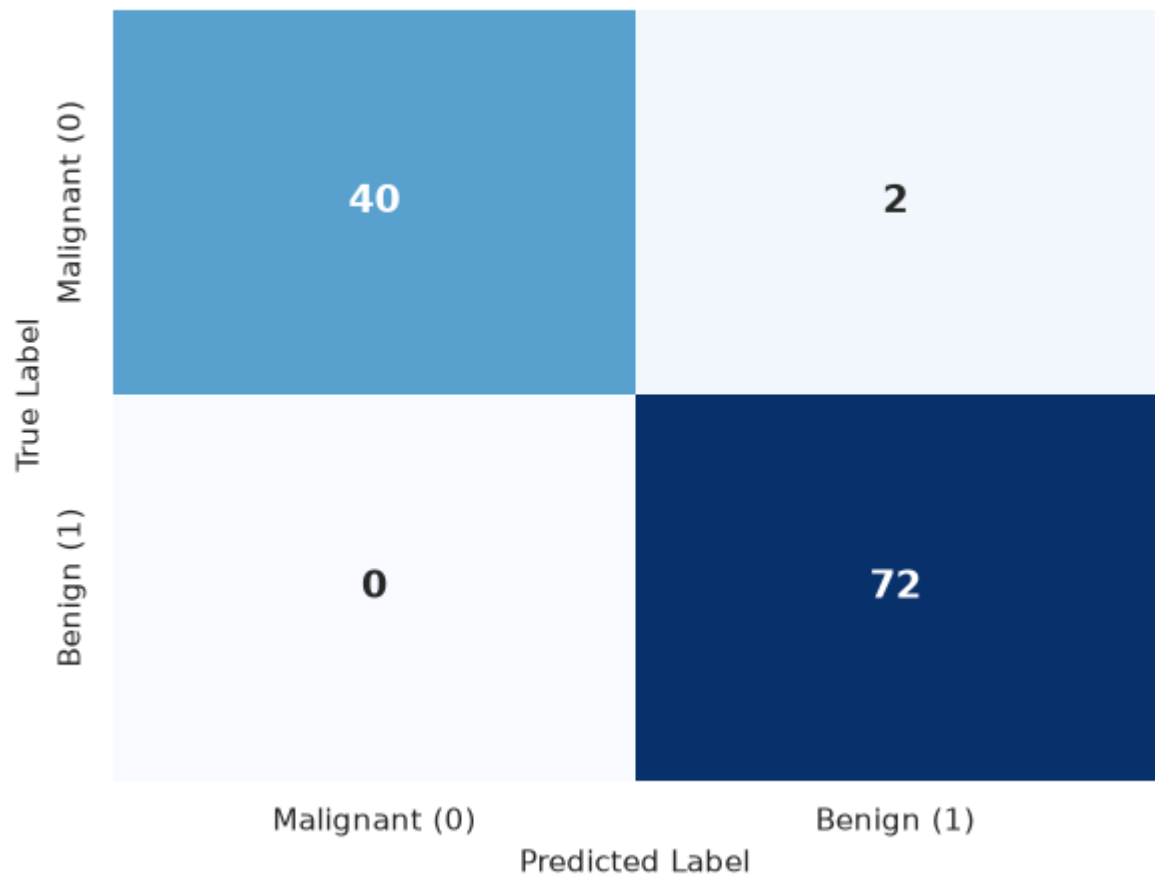
plt.figure(figsize=(8, 6), dpi=100)
plt.plot(fpr, tpr, color='#1D3557', lw=2, label=f'ROC Curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='#E63946', lw=2, linestyle='--', label='Random')
```

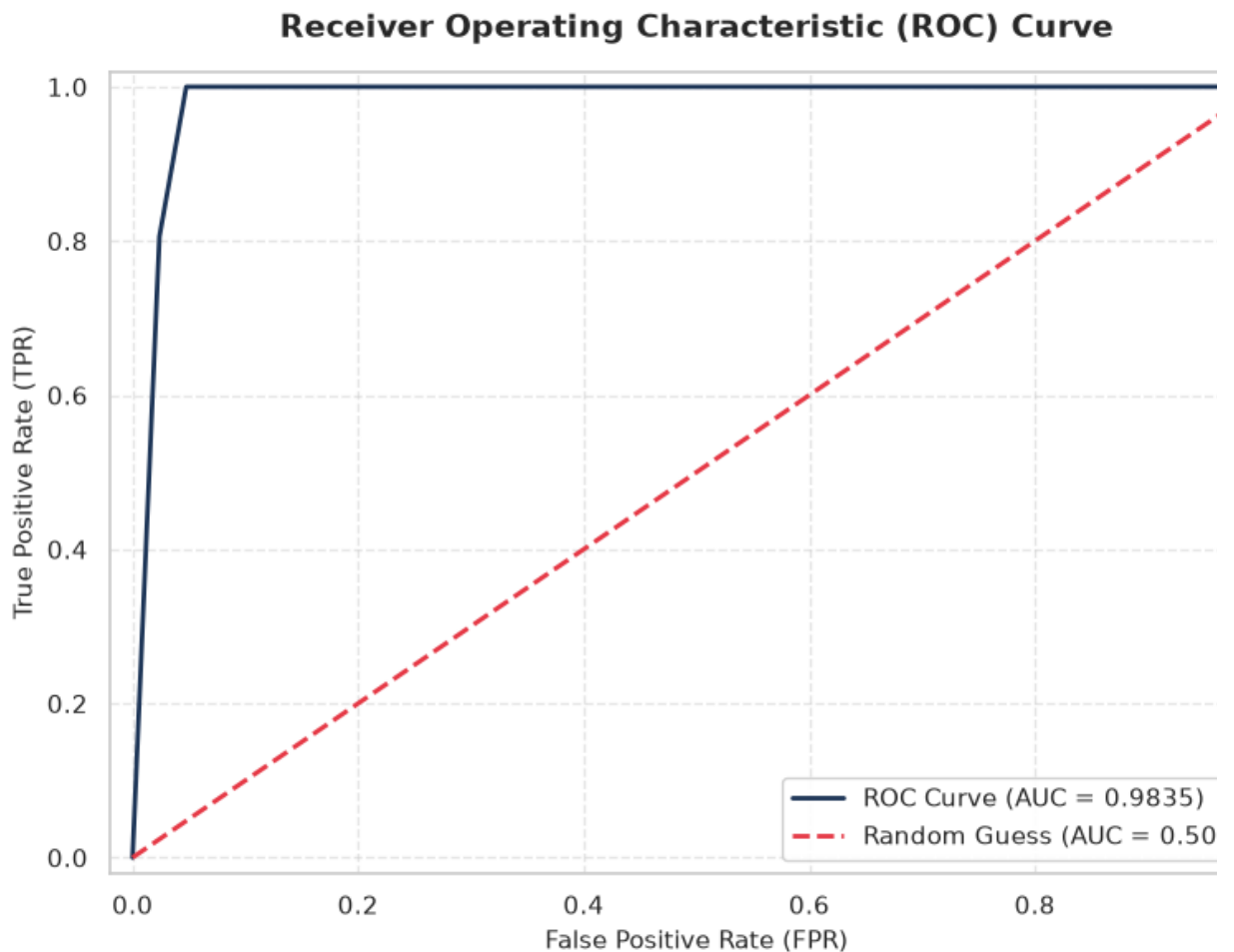
```
plt.xlim([-0.02, 1.02])
plt.ylim([-0.02, 1.02])
plt.title('Receiver Operating Characteristic (ROC) Curve', fontsize=14, f
plt.xlabel('False Positive Rate (FPR)', fontsize=11)
plt.ylabel('True Positive Rate (TPR)', fontsize=11)
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(loc='lower right', frameon=True, facecolor='white', framealpha
plt.tight_layout()
plt.savefig('../images/roc_curve.png', dpi=150)
plt.show()
```

--- Classification Report ---

	precision	recall	f1-score	support
Malignant (0)	1.00	0.95	0.98	42
Benign (1)	0.97	1.00	0.99	72
accuracy			0.98	114
macro avg	0.99	0.98	0.98	114
weighted avg	0.98	0.98	0.98	114

Confusion Matrix





Task 6 — Regression vs Classification

Evaluating model performance requires choosing the correct metric category.

Property	Regression (studied in Lab 3)	Classification (studied in Lab 4)
Output Type	Continuous numerical values	Discrete categorical classes
Objective	Minimize prediction error	Maximize classification correctness
Key Metrics	MAE, MSE, RMSE, R^2 Score	Accuracy, Precision, Recall, F1 Score, ROC-AUC
Fit Metric	R^2 measures explained variance	ROC-AUC measures class separation threshold independently

Detailed Comparisons

1. R^2 Score vs. Accuracy

- R^2 Score (Coefficient of Determination):** Measures the proportion of variance in the dependent variable explained by features, relative to a baseline mean predictor. Ranges from $-\infty$ to 1.0 .
- Accuracy:** Measures the raw percentage of correctly classified cases out of total predictions. Ranges from 0.0 to 1.0 (or 100%).

2. RMSE vs. F1 Score

- **RMSE (Root Mean Squared Error):** Measures the average prediction error, penalizing deviations quadratically. Expressed in the target's original units.
- **F1 Score:** The harmonic mean of Precision and Recall, balancing false positive rates and negative rates. Crucial for imbalanced categorical data.

3. MAE vs. Confusion Matrix

- **MAE (Mean Absolute Error):** Computes the average magnitude of absolute prediction errors, treating all deviations linearly.
- **Confusion Matrix:** A full contingency table breaking down correct and incorrect predictions across all classes, mapping specific errors (False Positives vs. False Negatives).

Analytical Questions

1. Why is KNN called a lazy learning algorithm?

KNN is called a **lazy learner** because it does not have a formal training phase where it fits mathematical weights or generates a structural model (like Coefficients in Linear Regression or Nodes in a Decision Tree). Instead, it simply stores (memorizes) the training instances. All computations—calculating distance to other points and performing majority voting—are deferred until a prediction is requested.

- **Training Time complexity:** $O(1)$
- **Inference Time complexity:** $O(N \cdot D)$ (where N is the number of samples, and D is features), which makes predictions computationally expensive.

2. Why is feature scaling necessary?

KNN calculates similarity between instances using distance measures (like Euclidean distance). If one feature has a wide numeric scale (e.g., `area_mean` ranging from 100 to 2500) and another feature has a tiny numeric scale (e.g., `smoothness_mean` ranging from 0.05 to 0.15), the calculation will be dominated by the larger feature. Feature scaling via `StandardScaler` scales all features to a mean of 0 and a standard deviation of 1, allowing them to contribute equally to the final prediction.

3. Explain heuristic K selection.

The heuristic $K = \sqrt{n}$ (where n is the number of training samples) provides a mathematically grounded baseline to balance the bias-variance trade-off.

- Setting K too small makes the classifier highly sensitive to local noise (high variance).
- Setting K too large makes the classifier ignore local boundaries, predicting based on majority classes (high bias). Taking the square root scale helps scale the neighborhood.

dynamically with data size. In practice, we adjust the output to the nearest odd integer to prevent tie-votes.

4. Why is cross validation more reliable?

A single train-test split could create a test set containing anomalously easy or hard instances leading to an inaccurate measure of generalization. Cross-validation (e.g., K-Fold) partitions the dataset into k equal segments, training the model k times while using a different segment as the validation set each time. Averaging these scores minimizes split-variance and utilizes every data point for both training and validation.

5. Explain bias-variance trade-off using K.

- **Low K (e.g., $K=1$):** High variance, low bias. The decision boundary is highly complex, adapting to every individual data point (overfitting). It captures local noise.
- **High K (e.g., $K=20$):** High bias, low variance. The decision boundary becomes smooth and simple, ignoring local variations (underfitting).
- **Optimal K :** The neighborhood size that minimizes the test error by balancing generalization and local resolution.

6. Why is Recall more important than Accuracy in cancer diagnosis?

In clinical cancer diagnosis:

- **False Positive (FP):** Predicting a healthy patient has cancer. This leads to anxiety and secondary testing but is quickly corrected.
- **False Negative (FN):** Predicting a cancer patient is healthy. This is dangerous and can lead to a lack of treatment, which is fatal. Recall measures the proportion of actual cancer patients detected: $\text{Recall} = \frac{TP}{TP + FN}$. By maximizing Recall, we minimize Negatives, ensuring no actual cancer patient is missed. Accuracy can be high even if the model misses a critical portion of malignant patients.

7. What happens when K becomes very large?

When K grows to approach the total number of training samples n , the classifier converges to almost all data points. In the extreme limit of $K = n$, the model ceases to recognize local patterns and always predicts the majority class of the dataset, regardless of features. This represents maximum bias and zero variance (underfitting).

Conclusion

This lab successfully demonstrates the K-Nearest Neighbours (KNN) classification algorithm on the Breast Cancer Wisconsin dataset. We prepared the data, mapped classes, and scaled features using `StandardScaler` to prevent feature dominance. Through experimenting with split sizes and parameter searches, we highlighted the trade-offs of split sizes and found the optimal values. Evaluating with cross-validation and classification metrics (Precision, Recall, F1 Score)

Confusion Matrix, ROC-AUC) highlighted the clinical importance of high Recall in cancer detection. Comparing these to regression metrics from Lab 3 demonstrates the fundamental differences between predicting continuous and categorical variables.

References

1. UCI Machine Learning Repository – Breast Cancer Wisconsin Diagnostic Dataset.
2. Scikit-Learn Documentation: `KNeighborsClassifier` and `StandardScaler`.
3. ROC Curves & AUC evaluation documentation.
4. Lab 3 – Linear Regression Evaluation Metrics.